

MetaCognition: 可验证的跨模型认知架构

产品需求文档 (PRD) + 技术规范 (Spec)

为 Claude Code 构建有状态的、跨会话的、多角色专家会诊系统。

版本: 1.0 | 日期: 2026-05-19 | 状态: 设计定稿

目录

- 1. 执行摘要
- 2. 问题陈述
- 3. 产品愿景
- 4. 架构总览
 - 4.1 三层认知架构
 - 4.2 与现有生态的关系
- 5. MetaCognition Skills (Skill 层)
 - 5.1 设计哲学
 - 5.2 Skill 1: record_draft_insight
 - 5.3 Skill 2: consult_external_brain
 - 5.4 Skill 3: promote_live_constraints
 - 5.5 决策门与触发条件
 - 5.6 用户交互模式
 - 5.7 错误处理与升级
- 6. Expert Brain MCP Server (协议层)
 - 6.1 系统架构
 - 6.2 MCP Tools 实现
 - 6.3 知识索引引擎
 - 6.4 外脑路由与缓存
 - 6.5 验证沙盒
 - 6.6 熔断与配额
- 7. 与 Superpowers 的融合
 - 7.1 融合定位
 - 7.2 逐阶段映射
 - 7.3 控制流时序
 - 7.4 数据流回路
 - 7.5 冲突仲裁规则
- 8. TDD 流程适配
 - 8.1 Red 阶段
 - 8.2 Green 阶段
 - 8.3 Refactor 阶段
- 9. 配置与部署
 - 9.1 完整配置示例
 - 9.2 Claude Desktop 配置
 - 9.3 启动时校验
- 10. 实施路线图
- 11. 附录
 - 11.1 术语表

1. 执行摘要

MetaCognition 是一套为 Claude Code 设计的认知增强架构，解决当前 AI 辅助编程中”瞬时经验流失”、“盲目重试”和”规则库膨胀”三大痛点。

它由两个互补的组件构成：

1. **MetaCognition Skills** — 声明式行为契约，定义主智能体何时、如何、向谁寻求外部认知支持，以及如何处理经验沉淀。以纯 prompt/JSON 契约形式存在于 Claude Code 的 SKILL.md 和 CLAUDE.md 中，**不包含任何实现细节**。
2. **Expert Brain MCP Server** — 基于 Model Context Protocol (MCP) 的协议层实现，封装外脑路由、语义缓存、两阶段验证、知识索引、Docker 沙盒、熔断配额等全部技术细节。对上层 Skill **完全透明**。

核心创新：

- **双态知识生命周期**：草稿态 (Draft) 高容错、轻量级捕获；发布态 (Live) 经严格验证后固化，支持自动晋升与衰减归档。
- **三层认知架构**：本地层 (Claude Code 主会话) → 检索层 (结构化知识索引) → 外脑层 (多角色 LLM 专家会诊)。
- **自适应复杂度路由**：Micro («5 行) 仅 Librarian 预检；Light («50 行) 选择性触发；Full (架构变更) 强制全量决策门。
- **与 Superpowers 深度融合**：在 Superpowers 的 6 阶段 workflow (Deliberation → Planning → Execution → Quality → Verification) 的每个转换节点插入认知守护。

2. 问题陈述

2.1 痛点一：瞬时经验流失

Claude Code 的 auto-memory 系统 (MEMORY.md) 在设计上会记录用户的纠正反馈，但社区实测表明它创造了一种**虚假的可靠感**——模型能”背诵”记忆内容，却在几分钟后再次犯同样的错误。根本原因是 LLM 的**指令层级冲突**：环境提示 (记忆) 在任务执行模式下被丢弃，记忆以被动文本形式注入上下文，权重不足。

2.2 痛点二：盲目猜测与无效重试

当模型遭遇超出其全局知识边界的深度技术屏障 (如死锁、并发竞争、破坏性重构) 时，倾向于在当前受限的上下文内连续盲目重试。既消耗 Token 又破坏代码树结构。现有工具 (如 Anthropic Advisor Tool) 是单次、无状态的咨询，无法沉淀跨会话的集体记忆。

2.3 痛点三：规则库过度膨胀

若将所有偶然踩坑的经验直接写入全局静态约束文件 (如 CLAUDE.md)，会导致主智能体在处理微小任务时背负过于沉重的负向检索包袱，进而产生幻觉或拒绝执行。Anthropic 的 MEMORY.md 存在约 25KB / 200 行的加载上限，旧记忆膨胀导致最新规则被静默截断。

3. 产品愿景

让 Claude Code 学会” 如何对待自己的无知”——不是变得更聪明，而是建立对自身认知缺陷的感知、求助与修复机制。

3.1 核心目标

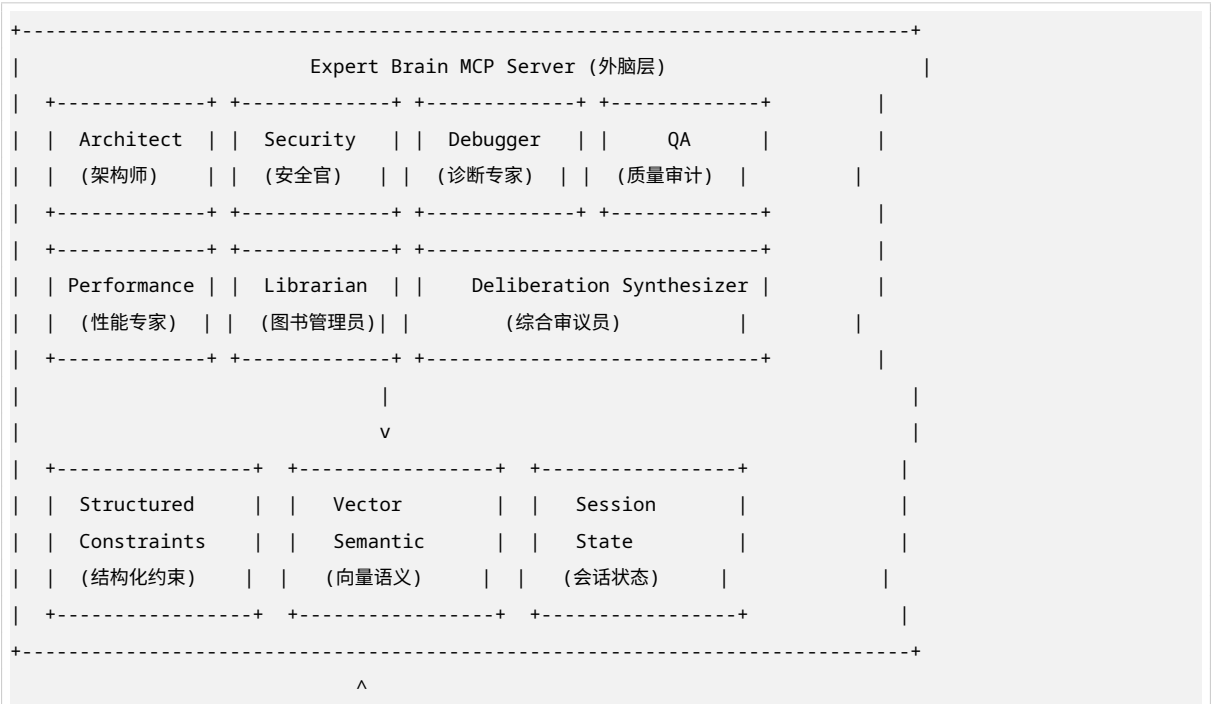
目标	度量	现状对比
消除重复踩坑	同一问题 24h 内重复出现率 < 5%	当前 auto-memory 无法防止
降低盲目重试	同一错误平均修复尝试次数 <= 2	当前常见 4-8 次
控制规则膨胀	CLAUDE.md 约束条目自动归档，活跃条目 < 50	当前无衰减机制
验证所有主张	外脑方案 100% 经过 sandbox 验证才固化	当前无强制验证

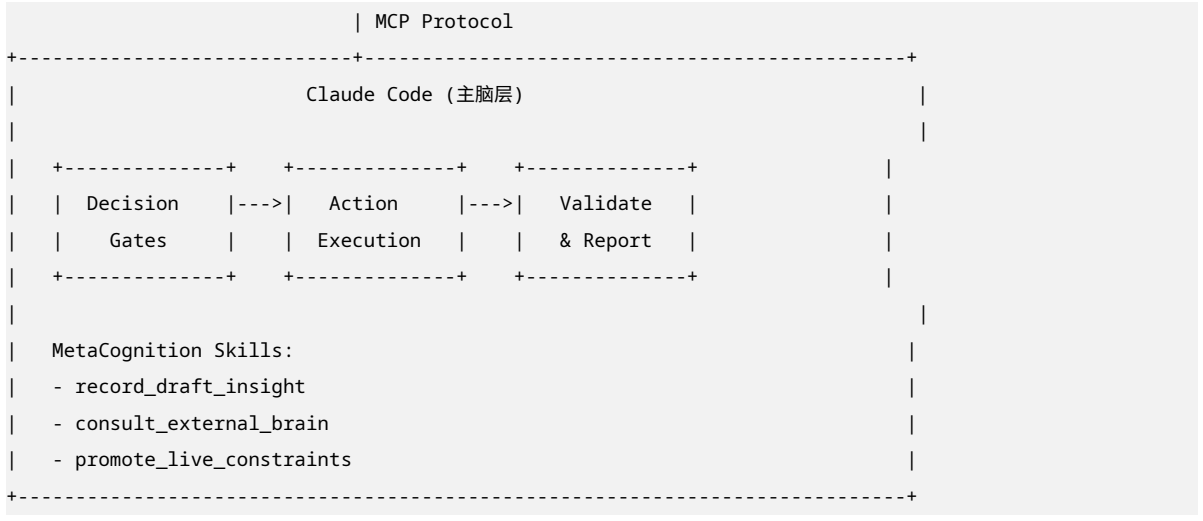
3.2 设计原则

- 1. **Process over Guessing**: 强制决策门，不允许模型绕流程。
- 2. **Evidence over Claims**: 所有外脑方案必须经过 sandbox 验证才能写入索引。
- 3. **Negative Knowledge First**: 优先检索”不能做什么”，再检索”应该怎么做”。
- 4. **Stateful over Stateless**: 咨询结果持久化，支持跨会话复利增长。
- 5. **Decay over Accumulation**: 经验会过时，约束会腐烂，系统必须支持归档而非无限膨胀。
- 6. **Flow over Friction**: 认知辅助应在背景运行，只在真正危险时拦截，不破坏 vibe coding 的直觉流。

4. 架构总览

4.1 三层认知架构





4.2 与现有生态的关系

组件	角色	MetaCognition 如何增强
Claude Code	主智能体	通过 MCP Tools 调用外脑，Skill 层植入决策门
Superpowers	工程 workflows 框架	在 6 个阶段转换节点插入认知守护，不替代任何技能
gstack	角色化决策框架	/plan-eng-review → Architect Gate; /cso → Security Gate
Anthropic	模型层协作原语	被 Expert Brain 内部作为 Architect/Security 的快速通道
Advisor Tool		
MCP Protocol	工具调用标准	全部外脑调用和知识操作通过 MCP 完成，禁止 Bash 子进程

5. MetaCognition Skills (Skill 层)

5.1 设计哲学

MetaCognition Skills 是纯声明式的行为契约，回答三个问题：

- **WHEN**：何时触发认知检查？
- **WHAT**：咨询谁？期望什么？
- **HOW**：结果如何消费？

绝对禁止出现在 Skill 层的内容：文件路径、Python 代码、SQL 语句、Bash 命令、API 密钥、Docker 配置。这些全部下沉到 Expert Brain MCP Server。

5.2 Skill 1: record_draft_insight

定位：无感知的背景动作或极低心流消耗的显式命令，用于捕获当前开发周期内被成功解决的诡异错误。属于”草稿态”阶段，具备高容错、轻量级特点。

触发条件：

- 用户显式说：“记住了”、“这是个陷阱”、“以后别踩这个坑”
- Bug 修复后，用户确认：“解决了，记下来”
- 自动触发（可选）：同一类错误在 24h 内第二次出现且被修复

行为契约:

```
## Skill: Record Draft Insight

**Trigger**: User explicitly signals a lesson learned, or auto-trigger on repeated bug pattern.

**Pre-condition**: The bug has been resolved and the fix is verified (tests pass or manual confirmation).

**Action**:
1. Synthesize a concise insight from the current context:
  - Symptom: What went wrong (user-facing description, 1 sentence)
  - Root Cause: Technical explanation (1-2 sentences)
  - Resolution: Verified fix or workaround (1 sentence)
  - Context Fingerprint: Affected files + git diff hash (auto-generated)
2. Call MCP Tool: `expert_brain/draft_insight`
3. Present result to user:
  - If "new": "Insight recorded. Hit count: 1"
  - If "updated": "Insight reinforced. Hit count: N"
  - If "duplicate": "This pattern is already tracked (ID: xxx)."

**Post-condition**: Insight exists in Draft state. It does NOT modify CLAUDE.md.

**Constraints**:
- NEVER write to local files directly.
- NEVER append to CLAUDE.md.
- NEVER execute Bash commands to manipulate storage.
```

5.3 Skill 2: consult_external_brain

定位：当主智能体在当前代码上下文中陷入困境，或需要突破跨领域架构边界时，挂起当前主心流，将处理权路由至专职外部推理专家。

触发条件（三级自适应）:

级别	条件	行为
Auto (Micro)	Librarian 在每次 tool call 前拦截，命中 high-severity 负向约束	阻断并提示，不咨询外脑
Auto (Full)	同一错误修复 >=2 次失败；或新增模块/依赖/API	强制咨询对应专家
Manual	用户说” 问问架构师”、“帮我审查安全”、“这个设计拿不准”	咨询用户指定的专家

行为契约:

```
## Skill: Consult External Brain

**Trigger**: Auto-gate failure, user request, or deliberation gate.

**Pre-condition**: Problem context is assembled (error logs, relevant code, attempted fixes).

**Action**:
```

```
1. Classify the problem domain:
- architecture → architect
- security / auth / input → security
- debugging / test failure / error log → debugger
- testing / coverage / boundary cases → qa
- performance / bottleneck / optimization → performance
- multi-domain conflict → deliberation
2. Call MCP Tool: `expert_brain/consult`
- expert: <domain>
- context: <assembled context>
- problem_type: <classification string>
- bypass_cache: false (default)
3. Receive structured proposal:
- If confidence >= 0.75 and validation_required == false:
  → Apply proposal directly, present to user for confirmation
- If validation_required == true:
  → Proceed to `expert_brain/validate`
- If confidence < 0.5:
  → Flag uncertainty, present raw proposal + disclaimer to user

**Post-condition**: A validated or acknowledged proposal exists.

**Constraints**:
- NEVER spawn subprocesses (python, curl, node).
- NEVER construct raw API requests.
- NEVER bypass the MCP Tool layer.
```

5.4 Skill 3: promote_live_constraints

定位：将”草稿态”的流动资产晋升为”发布态”的防御红线。通过定期复盘或显式命令触发。

触发条件：

- 用户显式说：“固化这条规则”、“把这个写进规范”
- 自动触发：insight hit_count >= 5（默认，可配置）AND 通过沙盒验证 AND 无冲突

晋升门槛（故意严格，避免噪音固化）：

条件	阈值	说明
Hit Count	>= 5	偶发环境问题不应被固化为永久约束
验证状态	已通过沙盒	未经运行的假设不能成为规则
冲突检测	无冲突	与现有 live constraints 不矛盾
时效性	3 个月内命中	过时经验应归档而非固化

行为契约：

```
## Skill: Promote Live Constraints

**Trigger**: User explicit command, or auto-trigger on mature validated insight.
```

```
**Pre-condition**: Target insight exists in Draft state with sufficient hit count and validation
↳ evidence.

**Action**:
1. Call MCP Tool: `expert_brain/promote`
  - insight_id: <target insight>
  - promotion_type: <negative_constraint | success_pattern | anti_pattern>
  - validation_evidence: <sandbox results, test output, reviewer approval>
2. Receive promotion result:
  - If "promoted":
    ↳ Present diff preview to user (what changed in CLAUDE.md / SQLite)
    ↳ If requires_human_confirm == true: wait for user "yes" before applying
  - If "conflict":
    ↳ Present conflicting rules, ask user to resolve or merge
  - If "insufficient_evidence":
    ↳ Explain what's missing, suggest running more tests
  - If "threshold_not_met":
    ↳ Inform user of current hit count and required threshold

**Post-condition**: Rule is promoted to Live state, or user is informed why not.

**Constraints**:
- NEVER directly modify CLAUDE.md.
- NEVER execute SQL INSERT statements.
- NEVER bypass the MCP Tool layer.
```

5.5 决策门与触发条件

借鉴 Superpowers 的 “IF A SKILL APPLIES... YOU MUST USE IT” 机制，以下 6 个操作前**必须**调用 Expert Brain:

决策门	触发条件	默认专家	自适应复杂度
Architecture Gate	新增模块、重大重构、跨服务调用、引入依赖	Architect	Full
Security Gate	用户输入处理、文件上传、权限变更、外部 API 调用	Security	Full
Debug Gate	同一错误修复 >=2 次失败、未知异常、性能衰退	Debugger	Full
QA Gate	新功能开发、复杂边界条件、历史 bug 区域	QA	Light/Full
Performance Gate	DB 查询修改、循环嵌套、缓存策略调整	Performance	Light
Anti-Pattern Gate	每次 tool call 前自动拦截 (仅 Librarian)	Librarian	Micro

自适应复杂度路由:

```
complexity_rules:
  micro:
```

```
condition: "lines_changed < 5 and files_touched == 1 and no_test_change and no_dep_change"
behavior: "librarian_only"      # 仅检查 negative constraints 索引

light:
condition: "lines_changed < 50 and files_touched <= 3"
behavior: "selective"          # 仅 security + qa (若触发条件满足)

full:
condition: "new_files > 0 or new_deps > 0 or api_surface_changed or architectural_impact"
behavior: "all_gates"          # 完整决策门流程
```

5.6 用户交互模式

记录洞察 (record__draft__insight)

用户说：“这是个坑，记住了” → Claude 调用 expert_brain/draft_insight → 返回：“已记录洞察 draft_0e41b9。当前命中次数：3。类似洞察：draft_a3f2, draft_b701”

咨询外脑 (consult__external__brain)

场景 A：自动触发 (Debug Gate) Claude 尝试修复同一 bash 语法错误 2 次失败：

- 1. 自动调用 expert_brain/consult (expert=debugger)
- 2. 返回提案：“使用 && 而非换行符连接命令”
- 3. Claude 应用提案，验证通过
- 4. 自动调用 expert_brain/draft_insight 记录该陷阱

场景 B：用户显式请求用户说：“这个重构方案拿不准，问问架构师”

- 1. 调用 expert_brain/consult (expert=architect, context= 重构计划)
- 2. 返回提案 + confidence=0.82 + validation_required=true
- 3. Claude 调用 expert_brain/validate
- 4. 验证通过后，向用户展示：“架构师建议：... (验证通过)”

固化规则 (promote__live__constraints)

用户说：“把那条规则写进规范”

- 1. 调用 expert_brain/promote (insight_id=draft_0e41b9)
- 2. 返回：“需要人工确认。CLAUDE.md 将新增： - 禁止在 bash 中使用换行符连接命令 [Promoted from draft_0e41b9]。确认? (yes/no)”
- 3. 用户确认后，Server 原子写入 CLAUDE.md

5.7 错误处理与升级

三层限制：

限制类型	阈值	行为
同问题重试上限	同一 query signature 24h 内最多 3 次	第 3 次失败后强制升级

Table 6 – continued

限制类型	阈值	行为
同专家连续失败	同一专家连续 2 次验证失败	自动切换专家角色
全局咨询配额	每会话最多 10 次外脑咨询	超出后仅允许 Librarian 本地查询

用户通报格式（当触发限制阈值时）：

```
## 🚨 MetaCognition 升级通知

** 问题签名 **: `bash_sandbox_permission_loop`

** 状态 **: 专家配额已耗尽 / 熔断器已打开

** 已尝试路径 **:
1. Debugger (DeepSeek-R1): 使用 heredoc → 沙盒不支持 → 失败
2. Architect (Claude-Opus): 重构为 Python 脚本 → import 路径错误 → 失败
3. Security (Claude-Opus): 使用 MCP filesystem tool → 当前会话不可用 → 失败

** 建议操作 **:
- [ ] 手动提供正确的 bash 语法
- [ ] 放宽约束：允许换行符并添加权限提示
- [ ] 切换模式：使用本地脚本文件替代内联 bash

** 索引状态 **: 3 条失败路径已记录，24h 内不会重复尝试相同方案。
```

6. Expert Brain MCP Server (协议层)

6.1 系统架构

Expert Brain MCP Server 是 MetaCognition Skills 的协议层实现，通过 MCP Protocol 向 Claude Code 暴露 5 个核心 Tools 和 5 个 Resources。

6.2 MCP Tools 实现

Tool 1: expert_brain/draft_insight 职责：接收洞察数据，执行语义去重（向量相似度 threshold 0.92，非 MD5），写入 draft_insights 索引。

输入 Schema：

```
{
  "symptom": "string — What went wrong (1 sentence)",
  "root_cause": "string — Technical explanation (1-2 sentences)",
  "resolution": "string — Verified fix or workaround (1 sentence)",
  "context_fingerprint": "string — git diff hash + affected files",
  "severity": "enum: [low, medium, high, critical]",
  "tags": ["array of domain tags"]
}
```

输出 Schema:

```
{
  "insight_id": "string",
  "status": "enum: [new, updated, duplicate]",
  "hit_count": "integer",
  "similar_insights": ["array of insight IDs"]
}
```

关键实现:

- 语义去重替代 MD5: 使用向量相似度 + 时间衰减
- 上下文指纹独立存储: 仅用于溯源, 不参与去重
- 关联追踪: 中度相似时创建 parent_id, 形成洞察家族

Tool 2: expert_brain/consult 职责: 路由到对应专家, 管理语义缓存, 执行外脑调用, 返回结构化提案。

输入 Schema:

```
{
  "expert": "enum: [architect, security, debugger, qa, performance, librarian, deliberation]",
  "context": "string — Full problem description",
  "problem_type": "string — Classification for signature hashing",
  "model_override": {
    "provider": "string (optional)",
    "model": "string (optional)",
    "base_url": "string (optional)",
    "api_key": "string (optional, ${ENV_VAR} syntax)"
  },
  "bypass_cache": "boolean — default false"
}
```

输出 Schema:

```
{
  "consultation_id": "uuid",
  "proposal": "string — actionable recommendation",
  "confidence": "number 0-1",
  "validation_required": "boolean",
  "estimated_tokens": "number",
  "cached": "boolean"
}
```

关键实现:

- 语义缓存: 基于向量相似度 + 24h 时效, 非 MD5 字符串匹配
- 配置层级: 调用参数 > 角色配置 > 全局 default, 支持运行时覆盖
- API KEY 安全: 仅支持 \${ENV_VAR} 语法, 明文密钥启动时拒绝加载
- 熔断感知: 每次调用记录到 consultation_history

Tool 3: expert_brain/validate 职责：执行两阶段验证（设计意图 + 沙盒执行 + 质量审计），返回结构化结果。

输入 Schema:

```
{
  "consultation_id": "string — ID from previous consult",
  "proposal": "string — The solution to validate",
  "validation_type": "enum: [sandbox, design_intent, quality, all] — default all"
}
```

输出 Schema:

```
{
  "validation_id": "string",
  "passed": "boolean",
  "stage_results": {
    "design_intent": { "passed": "bool", "notes": "string" },
    "sandbox": { "passed": "bool", "stdout": "string", "stderr": "string", "exit_code": "int" },
    "quality": { "passed": "bool", "side_effects": ["array of file paths"] }
  },
  "failure_reason": "string (if passed == false)"
}
```

三阶段验证详解:

1. **Design Intent Validator**: 独立子代理（轻量模型如 Sonnet）检查外脑方案是否真正匹配原始问题，防止“plan says ‘instead’ but implementer reads ‘alongside’”的设计漂移。
2. **Sandbox Executor**: Docker 隔离环境，运行外脑提供的验证命令，捕获 stdout/stderr/exit code，超时 60 秒，网络隔离。
3. **Quality Auditor**: 检查副作用（是否修改了预期外的文件）、性能退化、安全边界。

Tool 4: expert_brain/promote 职责：评估晋升资格，检测冲突，原子写入 CLAUDE.md 或 SQLite。

输入 Schema:

```
{
  "insight_id": "string — Reference to existing draft insight",
  "promotion_type": "enum: [negative_constraint, success_pattern, anti_pattern]",
  "validation_evidence": {
    "sandbox_passed": "boolean",
    "test_results": "string (optional)",
    "reviewer_approval": "boolean (optional)"
  }
}
```

输出 Schema:

```
{
  "promoted_id": "string",
  "target_location": "enum: [CLAUDE.md, sqlite_index, both]",
  "conflicts": ["array of conflicting rule IDs"],
  "requires_human_confirm": "boolean",
  "diff_preview": "string (markdown diff)"
}
```

```
}

```

关键实现:

- 晋升门槛: Hit Count >= 5 (非 2), 90 天时效性检查
- 冲突检测: 向量相似度 + LLM 逻辑矛盾判断
- 原子写入: tempfile + shutil.move 保证幂等性
- 人工确认: 有冲突或 critical 级别时必须用户确认

Tool 5: expert_brain/retrieve 职责: 检索知识索引, 支持混合检索 (结构化 + 向量 + 关键词)。

输入 Schema:

```
{
  "query": "string",
  "index_type": "enum: [success_patterns, negative_constraints, failed_paths, draft_insights, all] -
    ↳ default all",
  "top_k": "integer - default 3",
  "min_confidence": "number - default 0.75"
}
```

混合检索算法 (RRF: Reciprocal Rank Fusion):

1. 向量检索 (语义匹配) 放宽召回 top_k * 2
2. 关键词检索 (精确匹配) top_k * 2
3. RRF 融合排序
4. 过滤置信度 >= min_confidence

6.3 知识索引引擎

四层索引

索引层	对应 Superpowers	存储格式	持久性	更新策略
project_context	project-map.md	SQLite + 文本	长期	手动 + 自动检测 git-hash 变更
decision_journal	session-log.md	SQLite + 文本	长期	仅人工标记 [saved] 的决策
session_state	state.md	内存 + Redis	临时	会话结束丢弃
negative_constraints	known-issues.md	SQLite + 向量	长期	验证失败自动写入

SQLite Schema (核心表)

```
-- 草稿洞察表
CREATE TABLE draft_insights (
  id TEXT PRIMARY KEY,
  semantic_hash TEXT NOT NULL UNIQUE,
  symptom TEXT NOT NULL,
  root_cause TEXT NOT NULL,
  resolution TEXT NOT NULL,
```

```

context_fingerprint TEXT,
severity TEXT CHECK(severity IN ('low', 'medium', 'high', 'critical')),
tags JSON,
hit_count INTEGER DEFAULT 1,
first_hit TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
last_hit TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
status TEXT CHECK(status IN ('draft', 'promoted', 'archived')) DEFAULT 'draft',
parent_id TEXT REFERENCES draft_insights(id),
embedding BLOB,
source_session TEXT
);

-- Live 约束表
CREATE TABLE live_constraints (
    id TEXT PRIMARY KEY,
    source_insight_id TEXT REFERENCES draft_insights(id),
    type TEXT CHECK(type IN ('negative_constraint', 'success_pattern', 'anti_pattern')),
    content TEXT NOT NULL,
    context_tags JSON,
    severity TEXT CHECK(severity IN ('low', 'medium', 'high', 'critical')),
    confidence REAL CHECK(confidence BETWEEN 0 AND 0.95),
    hit_count INTEGER DEFAULT 1,
    success_count INTEGER DEFAULT 0,
    failure_count INTEGER DEFAULT 0,
    deprecated BOOLEAN DEFAULT FALSE,
    version INTEGER DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_validated TIMESTAMP,
    embedding BLOB
);

-- 失败路径表
CREATE TABLE failed_paths (
    id TEXT PRIMARY KEY,
    query_signature TEXT NOT NULL,
    expert TEXT NOT NULL,
    proposed_solution TEXT,
    failure_reason TEXT NOT NULL,
    validation_output TEXT,
    consult_count INTEGER DEFAULT 1,
    first_attempt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_attempt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    exhausted BOOLEAN DEFAULT FALSE
);

-- 咨询历史表 (审计 + 熔断)
CREATE TABLE consultation_history (
    id TEXT PRIMARY KEY,
    expert TEXT NOT NULL,
    model TEXT NOT NULL,
    provider TEXT NOT NULL,

```

```
problem_type TEXT,
confidence REAL,
validation_required BOOLEAN,
estimated_tokens INTEGER,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
session_id TEXT
);

-- 会话配额跟踪表
CREATE TABLE session_quotas (
    session_id TEXT PRIMARY KEY,
    total_consultations INTEGER DEFAULT 0,
    architect_count INTEGER DEFAULT 0,
    security_count INTEGER DEFAULT 0,
    debugger_count INTEGER DEFAULT 0,
    qa_count INTEGER DEFAULT 0,
    performance_count INTEGER DEFAULT 0,
    deliberation_count INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

自动维护任务

- 归档陈旧洞察: 超过 180 天未命中的 draft insights 自动标记为 archived
- Hit Count 衰减: 每 90 天未命中则减半 (最低 1), 防止僵尸规则累积
- 约束过时标记: 超过 90 天未验证的 live constraints 标记为待 review

6.4 外脑路由与缓存

多提供商支持矩阵

提供商	配置示例	适用场景	备注
anthropic	claude-opus-4-20260201	复杂架构、安全审查	支持 Advisor Tool 快速通道
openai	gpt-4.1	测试生成、文档编写	函数调用能力强
deepseek	deepseek-r1	深度调试、根因分析	推理链长, 需调高 timeout
google	gemini-2.5-pro	长上下文分析、性能优化	1M+ token 上下文
ollama	llama3.2:3b	本地检索、轻量咨询	零成本, 延迟 <200ms
lmstudio	qwen2.5-coder	离线环境、代码专用	需本地 endpoint

API KEY 安全设计

- 绝不硬编码: 配置文件中 api_key 只接受 \${ENV_VAR} 语法或空字符串, 明文密钥启动时拒绝加载
- 最小权限: 安全专家可配置独立的受限 KEY (如只读审计权限)
- 审计追溯: MCP Server 启动时记录各专家使用的 KEY 来源 (环境变量名)
- 本地模型豁免: Ollama/LMStudio 等本地 provider 自动跳过 KEY 校验

语义缓存策略 替代原方案中的 MD5 预检，采用基于向量相似度的语义缓存：

- 缓存键：基于问题类型 + 上下文向量嵌入
- 时效：24 小时
- 命中条件：向量距离 < 0.08 (cosine similarity > 0.92) + 同 expert + 同 problem__type

6.5 验证沙盒

Docker 配置

```
services:
  expert-brain-sandbox:
    image: expert-brain-sandbox:latest
    network_mode: none           # 无网络
    read_only: true              # 只读根文件系统
    tmpfs:
      - /tmp:noexec,nosuid,size=100m
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    cap_add:
      - CHOWN
    mem_limit: 512m
    cpu_quota: 50000
    pids_limit: 32
```

执行流程

1. 创建临时工作目录
2. 写入工作文件（测试代码、生产代码）
3. 构建 docker run 命令（网络 none、只读、资源限制）
4. 执行并捕获 stdout/stderr/exit code
5. 超时 60 秒自动 kill

6.6 熔断与配额

配额管理

配额类型	阈值	行为
每会话总咨询	10 次	超出后仅允许 Librarian 本地查询
每专家咨询	Architect:3, Security:3, Debugger:4, QA:2, Performance:2, Deliberation:2	超出后切换专家或等待
同问题重试	24h 内 3 次	第 3 次失败后熔断
Token 预算	默认 50K/次, Debugger 100K	超出后降级

熔断器

- 触发: 连续 3 次验证失败
- 冷却: 30 分钟
- 恢复: 半开状态, 允许 1 次试探
- 行为: 通知用户、降级为仅本地索引、禁用 Advisor Tool 快速通道

7. 与 Superpowers 的融合

7.1 融合定位

	Superpowers	MetaCognition
角色	骨架—定义工程工作流的阶段与技能	神经系统—在关节处感知疼痛并求助
核心问题	“How to build systematically?”	“How to know what I don’ t know?”
关系	定义 做什么 的顺序	在 转换节点 插入认知检查

7.2 逐阶段映射

Superpowers Phase	MetaCognition 嵌入点	动作
Deliberation	brainstorming 产出多方案后	consult deliberation —冲突方案综合审议
Planning	每个跨模块 task 后	consult architect —架构边界审查
Execution	执行前/中/后	Librarian 预检 → Debug Gate(2 次失败) → Security Gate
Quality (TDD)	Red/Green/Refactor 每个转换点	QA Gate → Architecture Gate → record_draft_insight → Performance Gate
Verification	完成声明前	validate(all) —Design Intent + Sandbox + Quality (强制)
Context	Session start / 定期维护	Librarian 检索约束 + 自动晋升 hit_count>=5 的洞察

7.3 控制流时序

```
[Session Start]
|
|→ Superpowers: context-management (load project-map)
|→ MetaCognition: Librarian retrieves top-10 negative constraints
|
▼
[Phase 1: Deliberation]
|
|→ Superpowers: premise-check (should this exist?)
|→ MetaCognition: Librarian pre-check (have we failed here before?)
|→ Superpowers: brainstorming (generate options)
|→ MetaCognition: IF options >=3 OR conflicts → consult deliberation
|→ Superpowers: deliberation (stakeholder perspectives)
```



```

└─▶ MetaCognition: IF perspectives diverge → consult deliberation
|
▼
[Phase 2: Planning]
|
└─▶ Superpowers: writing-plans (decompose to micro-tasks)
└─▶ MetaCognition: FOR each cross-module task → consult architect
    └─▶ IF architect rejects → REPLAN
|
▼
[Phase 3: Execution]
|
└─▶ MetaCognition: Librarian pre-action check (auto, no latency)
└─▶ Superpowers: executing-plans (implement task)
└─▶ MetaCognition: IF same error 2x → consult debugger
└─▶ MetaCognition: IF security-sensitive → consult security
└─▶ Superpowers: subagent-driven-development (two-stage review)
└─▶ MetaCognition: QA generates review checklist
    └─▶ MetaCognition: IF reviewers conflict → consult deliberation
|
▼
[Phase 4: Quality]
|
└─▶ TDD Red: MetaCognition QA Gate (if test construction stalls)
└─▶ TDD Green: MetaCognition Architecture Gate (if cross-module)
└─▶ TDD Green-Refactor: record_draft_insight (surprising passes)
    └─▶ TDD Refactor: MetaCognition Performance Gate (if regression)
|
▼
[Phase 5: Verification]
|
└─▶ Superpowers: verification-before-completion (self-check)
    └─▶ MetaCognition: expert_brain/validate (ALL) — MANDATORY for Full
|
▼
[Phase 6: Context Update]
|
└─▶ Superpowers: context-management (update project-map, session-log)
└─▶ MetaCognition: auto-promote constraints (hit_count >=5)
    └─▶ MetaCognition: archive stale insights (>180 days)

```

7.4 数据流回路

正向流: Superpowers 产出 → MetaCognition 消费

- brainstorming 候选方案 → deliberation expert input
- writing-plans task list → architect expert input
- executing-plans error log → debugger expert input
- TDD test code → qa expert input

- verification diff → validate tool input

反向流: MetaCognition 产出 → Superpowers 消费

- negative_constraints → premise-check input
- architect proposal → writing-plans feedback
- debugger proposal → executing-plans input
- validate report → verification-before-completion gate
- promoted constraints → context-management project-map

7.5 冲突仲裁规则

冲突场景	优先级	行为
Librarian 拦截 vs 执行意愿	Librarian 优先	阻断执行，要求显式 override
Architect 拒绝 vs Plan 任务	Architect 优先	回流到 Planning，重新分解任务
Validator 失败 vs 完成声明	Validator 优先	禁止完成，回流到 Execution
配额耗尽 vs 流程要求	降级模式	仅本地检索，外脑咨询暂停

8. TDD 流程适配

TDD 是 MetaCognition 的最佳宿主，因为 TDD 的核心循环（Red → Green → Refactor）天然具备验证边界清晰、失败状态可观测、重构风险集中三个特征。

8.1 Red 阶段：测试构造困境

传统 TDD：硬想 10 分钟，写出脆弱的测试，后面反复修改

MetaCognition TDD：2 次尝试失败后，自动触发 consult_external_brain(QA)

```
WHEN writing a test:
  1. Attempt to express the behavior in test code
  2. IF the test is ambiguous, overly complex, or relies on implicit state → Retry once
  3. IF second attempt still fails to capture intent cleanly:
    → CALL expert_brain/consult (expert="qa", problem_type="test-construction")
    → Present QA expert's proposal: "Try Arrange-Act-Assert with Builder pattern"
  4. Apply proposal, record result
  5. IF test now cleanly fails for the right reason:
    → CALL expert_brain/draft_insight (symptom="unclear boundary expression", ...)
```

8.2 Green 阶段：最小代码的架构越界

传统 TDD：直接 import 外部服务，测试通过但架构腐化

MetaCognition TDD：触发 Architecture Gate，咨询 Architect

```
WHEN writing production code to make test pass:
  1. IF the minimal solution requires:
    - New dependency import
```

```
- Cross-module direct call
- Bypassing existing abstraction
→ PAUSE
2. CALL expert_brain/consult (expert="architect", problem_type="tdd-boundary-crossing")
3. Architect returns: "Introduce Repository interface instead of direct DB access"
4. Apply interface + mock, 保持测试通过
5. CALL expert_brain/draft_insight (root_cause="direct DB access in unit test")
```

8.3 Refactor 阶段：跨测试影响与性能退化

传统 TDD: 手动修改 12 个文件, 漏掉 2 个, CI 失败

MetaCognition TDD: 重构前咨询 Architect, 重构中 Librarian 拦截负向约束

```
BEFORE refactoring:
1. CALL expert_brain/retrieve (query="refactoring [method_name] test impact")
2. Librarian returns: "3 historical insights about this method's test fragility"
3. IF high-severity negative constraint hit:
    → "WARNING: Previous refactor broke 8 tests due to implicit dependency"
    → Require explicit override to proceed

DURING refactoring:
1. IF test suite execution time increases >20%:
    → CALL expert_brain/consult (expert="performance", problem_type="test-suite-regression")
2. IF cross-cutting change affects >3 test files:
    → CALL expert_brain/consult (expert="architect", problem_type="refactor-test-coupling")
```

9. 配置与部署

9.1 完整配置示例

```
# expert-brain.yaml
version: "1.0"

default:
  provider: "anthropic"
  model: "claude-sonnet-4-20250514"
  base_url: ""
  api_key: ""
  api_key_env: "ANTHROPIC_API_KEY"
  temperature: 0.2
  max_tokens: 800
  timeout_ms: 30000
  max_retries: 2

experts:
  architect:
    name: "Architect"
    model:
      provider: "anthropic"
      model: "claude-opus-4-20260201"
```

```

    base_url: ""
    api_key: "${ANTHROPIC_OPUS_KEY}"
    temperature: 0.3
    max_tokens: 1200
    advisor_tool:
        enabled: true
        max_uses: 3
    triggers: ["new_module", "major_refactor", "cross_service_call", "dependency_addition"]

security:
    name: "Security"
    model:
        provider: "anthropic"
        model: "claude-opus-4-20260201"
        base_url: ""
        api_key: "${ANTHROPIC_SECURITY_KEY}"
        temperature: 0.1
        max_tokens: 1000
    triggers: ["user_input_handling", "file_upload", "permission_change", "external_api_call"]

debugger:
    name: "Debugger"
    model:
        provider: "deepseek"
        model: "deepseek-r1"
        base_url: "https://api.deepseek.com/v1"
        api_key: "${DEEPSEEK_API_KEY}"
        temperature: 0.6
        max_tokens: 2000
        reasoning_effort: "high"
        timeout_ms: 60000
    triggers: ["test_failure_repeat:2", "error_unknown", "performance_regression"]

qa:
    name: "QA"
    model:
        provider: "openai"
        model: "gpt-4.1"
        base_url: ""
        api_key: ""
        temperature: 0.3
        max_tokens: 1000
    triggers: ["new_feature", "complex_boundary_conditions", "historical_bug_area"]

performance:
    name: "Performance"
    model:
        provider: "google"
        model: "gemini-2.5-pro"
        base_url: ""
        api_key: "${GOOGLE_API_KEY}"

```

```
    temperature: 0.2
    max_tokens: 1000
    triggers: ["db_query_change", "loop_nesting", "cache_strategy_change"]

librarian:
  name: "Librarian"
  model:
    provider: "ollama"
    model: "llama3.2:3b"
    base_url: "http://localhost:11434"
    api_key: ""
    temperature: 0.0
    max_tokens: 500
  auto_trigger:
    enabled: true
    intercept_mode: "pre_action"
    check_latency_ms: 200

deliberation:
  name: "Deliberation Synthesizer"
  model:
    provider: "anthropic"
    model: "claude-opus-4-20260201"
    base_url: ""
    api_key: ""
    temperature: 0.4
    max_tokens: 1500
  triggers: ["deliberation_gate"]

quotas:
  per_session:
    total_consultations: 10
  per_expert:
    architect: 3
    security: 3
    debugger: 4
    qa: 2
    performance: 2
    deliberation: 2
  per_query_signature:
    max_retries: 3
    retry_window_hours: 24
  token_budgets:
    default_per_consultation: 50000
    debugger: 100000
  circuit_breaker:
    failure_threshold: 3
    recovery_timeout_minutes: 30

validation:
  sandbox:
```

```

    type: "docker"
    image: "expert-brain-sandbox:latest"
    timeout_seconds: 60
    network: "none"
  auto_index_update:
    on_success: true
    on_failure: true

  promotion:
    hit_count_threshold: 5
    stale_days: 90
    archive_days: 180
    hit_count_decay_interval: 90
    decay_factor: 0.5

  escalation:
    notify_user_on:
      - "exhausted_retries"
      - "circuit_breaker_open"
      - "budget_exceeded"
      - "expert_disagreement_high"

```

9.2 Claude Desktop 配置

```

{
  "mcpServers": {
    "expert-brain": {
      "command": "python",
      "args": ["/path/to/expert-brain-server/main.py"],
      "env": {
        "EXPERT_BRAIN_CONFIG": "/path/to/expert-brain.yaml",
        "ANTHROPIC_API_KEY": "sk-ant-...",
        "ANTHROPIC_OPUS_KEY": "sk-ant-...",
        "ANTHROPIC_SECURITY_KEY": "sk-ant-...",
        "OPENAI_API_KEY": "sk-...",
        "DEEPSEEK_API_KEY": "sk-...",
        "GOOGLE_API_KEY": "...",
        "DATABASE_PATH": "/path/to/expert-brain.db"
      }
    }
  }
}

```

9.3 启动时校验

Server 启动时执行以下校验，任一失败则拒绝启动：

1. 配置文件格式校验：YAML 语法正确
2. **API KEY 明文检测**：禁止硬编码，只接受 `${ENV_VAR}` 语法
3. 环境变量可解析性：所有引用的环境变量必须存在
4. 数据库连接：SQLite 可读写

5. **Docker 可用性**: 如果启用沙盒, docker 命令必须可用

10. 实施路线图

Phase 1: 立法规制 (Week 1-2)

- 1. 在项目根目录创建 CLAUDE.md, 植入 MetaCognition 系统 hook
- 2. 配置 Claude Desktop 接入 Expert Brain MCP Server
- 3. 提取当前最严重的 5-10 个错误模式, 转化为 MUST/NEVER 规则
- 4. 配置 Librarian 自动拦截 (Ollama 本地部署)

Phase 2: 建立陷阱库 (Week 3-4)

- 1. 部署 expert_brain/draft_insight 和 expert_brain/retrieve
- 2. 每次 Claude 犯错且规则无法覆盖时, 记录到 draft_insights
- 3. 建立 “Known Traps” 分类体系

Phase 3: 接入外脑 (Week 5-6)

- 1. 部署 expert_brain/consult, 配置多提供商 (Anthropic/DeepSeek/OpenAI)
- 2. 部署 expert_brain/validate, 配置 Docker 沙盒
- 3. 建立验证流水线 (Design Intent + Sandbox + Quality)

Phase 4: 闭环自动化 (Week 7-8)

- 1. 部署 expert_brain/promote, 实现自动晋升
- 2. 配置 Hit Count 衰减与归档任务
- 3. 与 Superpowers 工作流全面融合

Phase 5: TDD 深度适配 (Week 9-10)

- 1. 在 TDD Red/Green/Refactor 各阶段植入决策门
 - 2. 建立测试构造知识库 (test-construction patterns)
 - 3. 建立重构安全规则库 (refactor safety constraints)
-

11. 附录

11.1 术语表

术语	定义
MetaCognition	主智能体对自身认知过程的管理: 感知无知、求助外脑、固化经验
Expert Brain	基于 MCP 的外脑服务层, 封装多模型路由、缓存、验证、索引
Decision Gate	强制咨询点, 不可绕过
Query Signature	问题哈希标识, 用于重试计数和熔断
Negative Constraint	“绝对不能做”的规则
Success Pattern	验证通过的解决方案模板

Table 13 – continued

术语	定义
Anti-Pattern	已证伪的尝试方案
Draft Insight	草稿态知识，高容错、轻量级捕获
Live Constraint	发布态规则，经严格验证后固化到 CLAUDE.md 或索引
Two-Stage Validation	设计意图验证 + 沙盒执行验证 + 质量审计
Deliberation	多专家并行咨询 + 综合审议
Advisor Tool	Anthropic server-side 模型协作原语 (Sonnet 执行 + Opus 顾问)
Anti-Pattern Gate	每次 tool call 前的自动负向约束检查
Circuit Breaker	连续失败后熔断，防止资源浪费
Semantic Cache	基于向量相似度的缓存，替代 MD5 字符串匹配
Context Fingerprint	代码上下文哈希 (git diff + 受影响文件)，用于溯源
Hit Count Decay	长期未命中的洞察自动降低权重，防止僵尸规则

11.2 参考文献

- Anthropic. *Claude Code Auto-Memory & Advisor Tool*. 2026.
- obra. *Superpowers Framework*. GitHub, ~150K stars.
- gstack. *Role-Based Decision Gates for AI Agents*. 2026.
- Model Context Protocol (MCP) Specification. *Protocol 2025-11*.
- DreamOS. *Biologically-Inspired Memory Architecture for LLM Agents*. EvolvingAgents Labs.
- /wizard Skill. *Pre-Action Checklist for Claude Code*. Dev.to, 2026.
- Self-Correction Loop. *Systematic Debugging & Memory Audit for .NET Agents*. 2026.
- dead-letter-oracle. *Governed MCP Agent with Closed-Loop Reasoning*. 2026.
- claude-concilium. *Multi-Agent AI Consultation Framework via MCP*. 2026.

文档版本: 1.0

最后更新: 2026-05-19

作者: AI Architecture Team

兼容: Claude Code Desktop 0.9+, MCP Protocol 2025-11, Superpowers 2026